If you're familiar with legacy .NET Framework ASP.NET or earlier versions of ASP.NET Core, you may already be familiar and comfortable with MVC controllers. If you're unsure whether to stick to what you know or switch to Razor Pages, this section should help you choose. Developers coming from those backgrounds often have misconceptions about Razor Pages initially (as I did!), incorrectly equating them with Web Forms and overlooking their underlying basis of the MVC framework. This section attempts to set the record straight.

Indeed, architecturally, Razor Pages and MVC are essentially equivalent, as they both use the MVC design pattern. The most obvious differences relate to where the files are placed in your project, as I discuss in the next section.

19.5.1 The benefits of Razor Pages

In section 19.5 I showed that the code for an MVC controller looks similar to the code for a Razor Page `PageModel`. If that's the case, what benefit is there to using Razor Pages? In this section I discuss some of the pain points of MVC controllers and how Razor Pages attempts to address them.

Razor Pages are not Web Forms

A common argument I hear from existing ASP.NET developers against Razor Pages is "Oh, they're just Web Forms." That sentiment misses the mark in many ways, but it's common enough that it's worth addressing directly.

Web Forms was a web-programming model that was released as part of .NET Framework 1.0 in 2002. It attempted to provide a highly productive experience for developers moving from desktop development to the web for the first time.

Web Forms are much maligned now, but their weaknesses only became apparent later. Web Forms attempted to hide the complexities of the web from you, to give you the impression of